13.4 billion tokens in 8 days.

Roughly **5–7 sessions**, running nearly **24/7**.

1.67B tokens per day on average

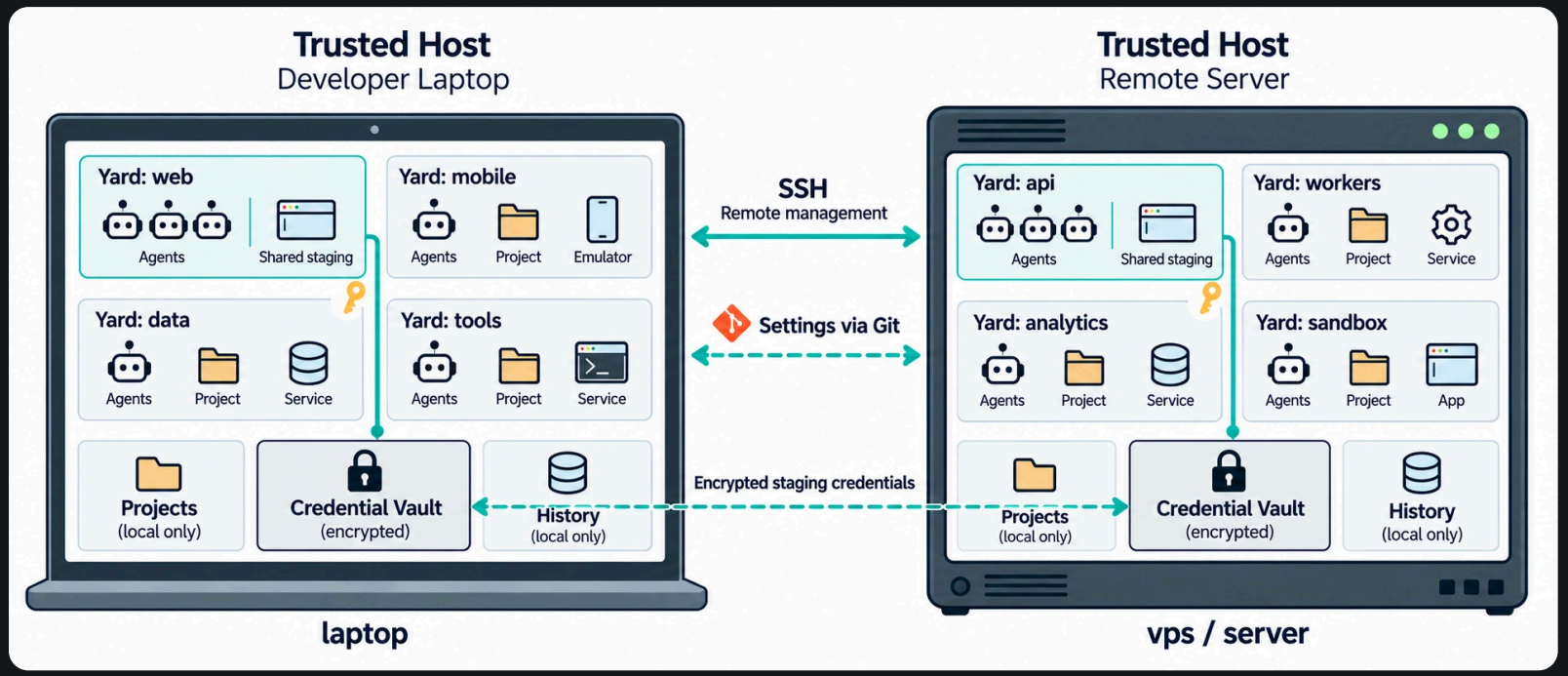
2026-10-05	All		22,920,885	3,171,017	0	1,025,325,184	1,051,417,086	$166.98
	- Codex	- gpt-6-luna - gpt-6.1-sol	22,920,885	3,171,017	0	1,025,325,184	1,051,417,086	$166.98
2026-10-06	All		38,080,225	5,801,484	0	1,847,434,368	1,891,316,077	$266.38
	- Codex	- gpt-6-luna - gpt-6.1-sol	38,080,225	5,801,484	0	1,847,434,368	1,891,316,077	$266.38
2026-10-07	All		27,697,279	4,168,198	0	1,092,886,016	1,124,751,493	$139.52
	- Codex	- gpt-6-luna - gpt-6.1-sol	27,697,279	4,168,198	0	1,092,886,016	1,124,751,493	$139.52
2026-10-08	All		42,544,619	6,976,510	0	1,896,151,680	1,945,672,809	$226.01
	- Codex	- gpt-6-luna - gpt-6.1-sol	42,544,619	6,976,510	0	1,896,151,680	1,945,672,809	$226.01
2026-10-09	All		43,408,079	7,558,899	0	1,848,873,984	1,899,840,962	$206.66
	- Codex	- gpt-6-luna - gpt-6.1-sol	43,408,079	7,558,899	0	1,848,873,984	1,899,840,962	$206.66
2026-10-10	All		40,899,666	6,482,878	0	1,472,235,392	1,519,617,936	$197.38
	- Codex	- gpt-6-luna - gpt-6.1-sol	40,899,666	6,482,878	0	1,472,235,392	1,519,617,936	$197.38
2026-10-11	All		43,459,370	7,520,475	0	1,706,033,280	1,757,013,125	$256.82
	- Codex	- gpt-6-luna - gpt-6.1-sol	43,459,370	7,520,475	0	1,706,033,280	1,757,013,125	$256.82
2026-10-12	All		40,088,969	6,887,092	0	1,426,293,376	1,473,269,437	$219.85
	- Codex	- gpt-6-luna - gpt-6-sol - gpt-6.1-sol	40,088,969	6,887,092	0	1,426,293,376	1,473,269,437	$219.85
2026-10-13	All		42,562,493	8,266,719	0	1,734,050,304	1,784,879,516	$310.66
	- Codex	- gpt-6-astra - gpt-6-luna - gpt-6.1-sol	42,562,493	8,266,719	0	1,734,050,304	1,784,879,516	$310.66

Original screenshot · October 5 is shown but excluded from the headline.

Source: supplied Codex usage screenshot · daily totals counted once

Session count and operating pattern: user estimate

Subyard Workspaces for coding agents.



Architecture diagram from the Subyard README.

Isolated agent environments

Containers or VMs with per-yard CPU and memory limits.

Shared staging

Run services and test changes on your own compute.

Across your machines

Manage hosts over SSH and sync settings through Git.

github.com/subyard/subyard

Source: Subyard README

Reference environment for this work

What each harness records and exports.

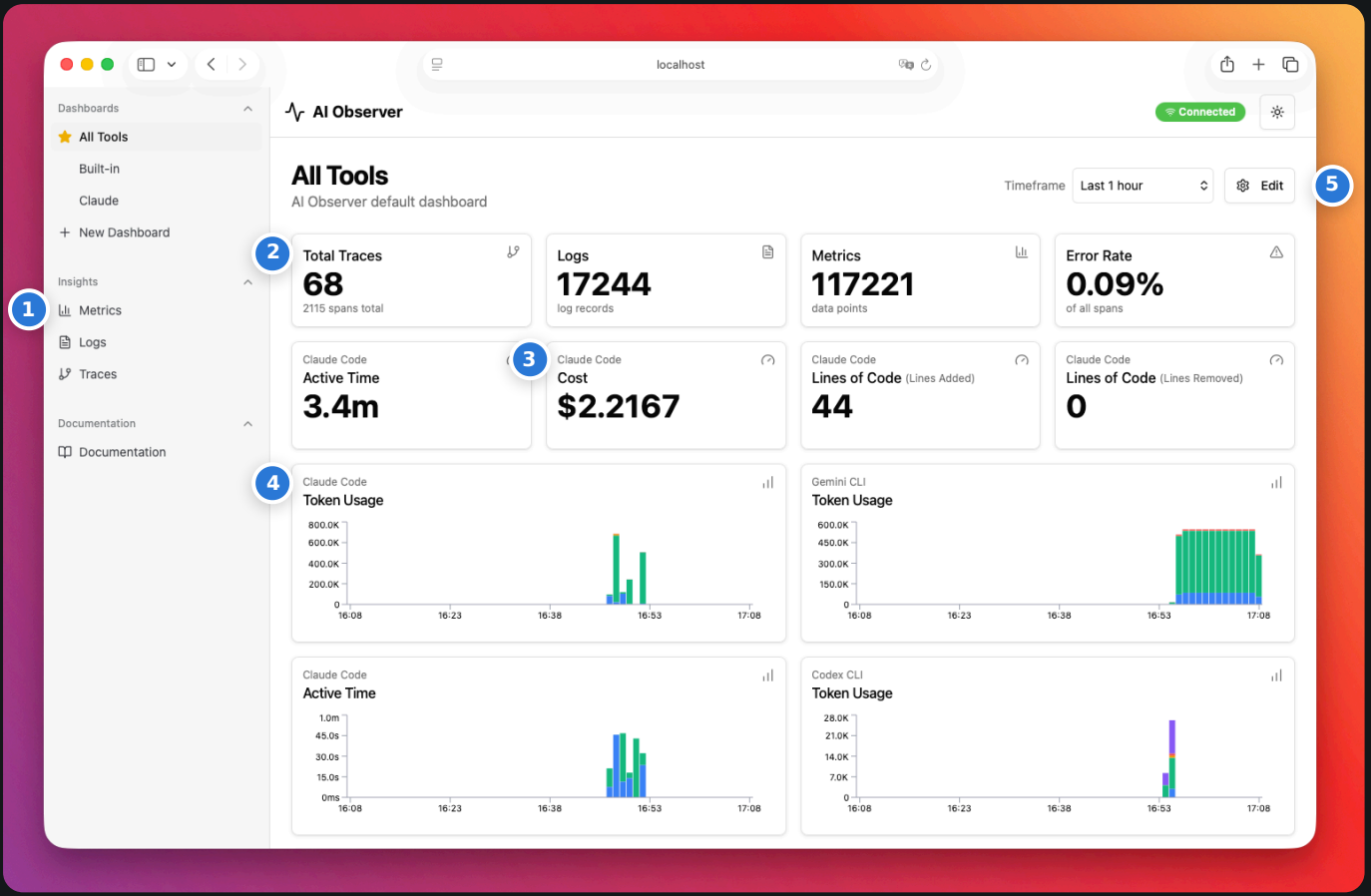
Our comparison categories, not vendor-defined levels. Metrics, events and traces are all telemetry.

Collection method	Codex CLI	Claude Code	OpenCode
01 / LOCAL **Saved by default** **Read after a session**	**Messages, tool calls/results, timestamps and token usage.** Input, cached input, output and reasoning counts in local rollouts. `~/.codex/ · local session files`	**Messages, tool calls/results and token/cost statistics.** JSONL transcripts, including subagents. Configurable cleanup; default 30 days. `~/.claude/ · transcripts + stats`	**Sessions, messages, tool inputs/results, tokens and cost.** Usage fields include input/output, cache read/write and reasoning.* `~/.local/share/opencode/`
02 / EXPORT **Metrics & events** **Configure collection**	**Token counts; API/tool durations, outcomes and approvals.** Configure OTel log and metric exporters plus the collector endpoint. `otel.exporter · otel.metrics_exporter`	**Tokens/cache, cost, active time, API calls and tool events.** Enable telemetry; select metric/log exporters and the collector endpoint. `CLAUDE_CODE_ENABLE_TELEMETRY=1`	**JSON session export and token/cost summaries.** Live collection needs an SDK/plugin adapter. Native OTLP metrics/logs are not documented. `opencode export · opencode stats`
03 / TRACING **Timed spans** **Optional telemetry: what ran, for how long**	**Native OTLP trace exporter.** Enable separately and set the destination. Actual span coverage needs a real-run check. `otel.trace_exporter`	**Interaction → API / tool spans** Beta Includes nested subagent calls. Enable enhanced telemetry and a trace exporter. `Beta flag + trace exporter`	**Native trace export: unverified.** Plan an SDK/plugin tracing adapter; validate span timing and parent/child links. `Custom instrumentation to evaluate`

Our integration work Normalize these records; correlate task → agent → step; add process CPU/RAM measurements.

OTel = OpenTelemetry; OTLP = its export protocol. Local records and collector exports are separate. *OpenCode usage fields checked in development source; verify the installed version. Provider analytics have separate defaults.

Activity, cost and tokens. One dashboard.



Official AI Observer example · the image underlying the supplied annotated slide

1 Explore the data
Metrics, Logs and Traces in the sidebar.

2 Start with the overview
Trace count, log records, metric points and error rate.

3 Read service metrics
Claude Code active time, cost and lines of code.

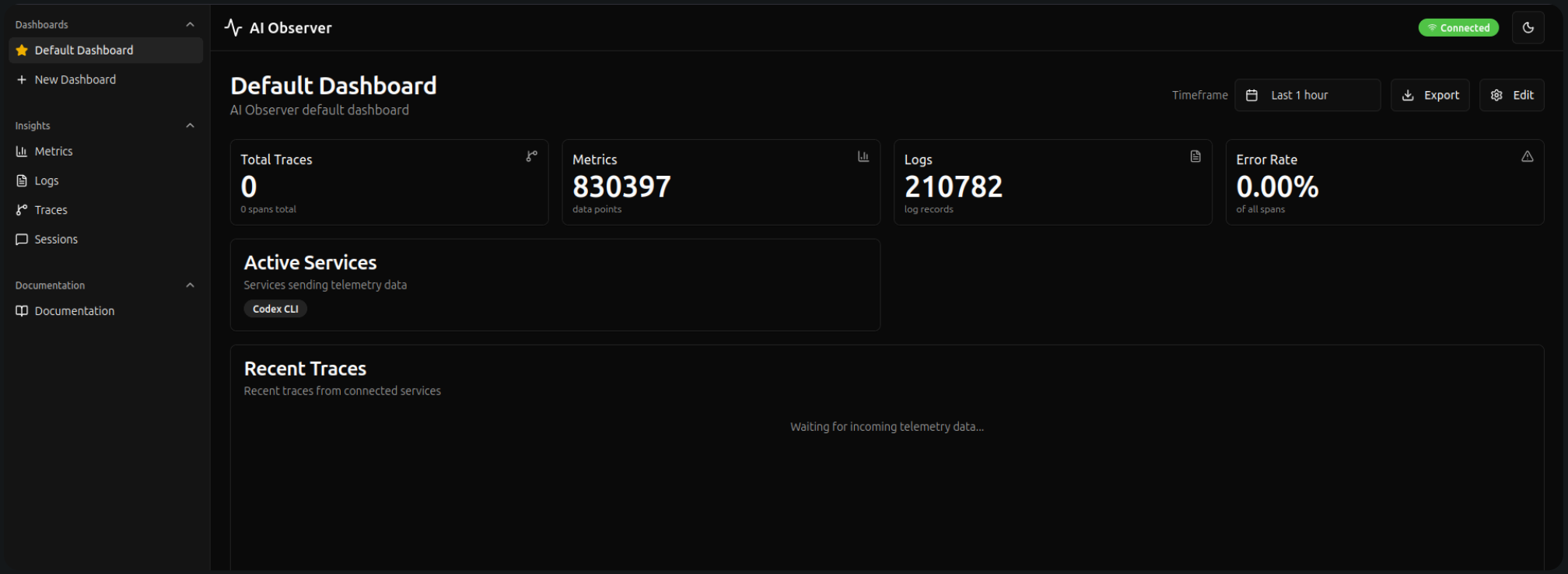
4 See usage over time
Token charts for Claude Code, Gemini CLI and Codex CLI.

5 Choose the view
Timeframe and dashboard editing controls.

Source: official AI Observer dashboard image, reused from presentation 2

Example data · last 1 hour

AI Observer Default Dashboard.

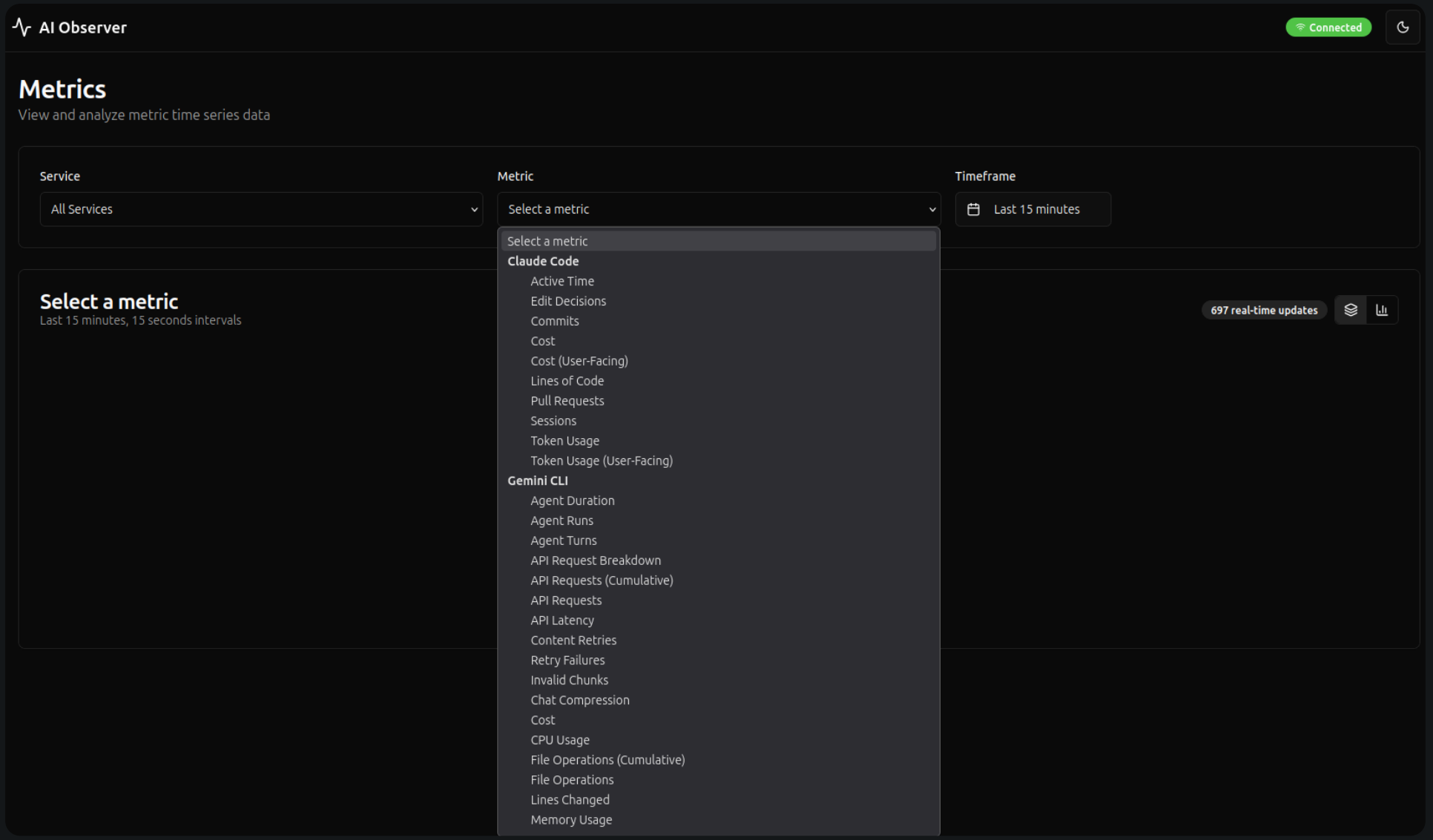


Metrics and logs are present; no traces are shown. With zero spans, the displayed error rate does not establish error-free operation.

Source: user-supplied original screenshot

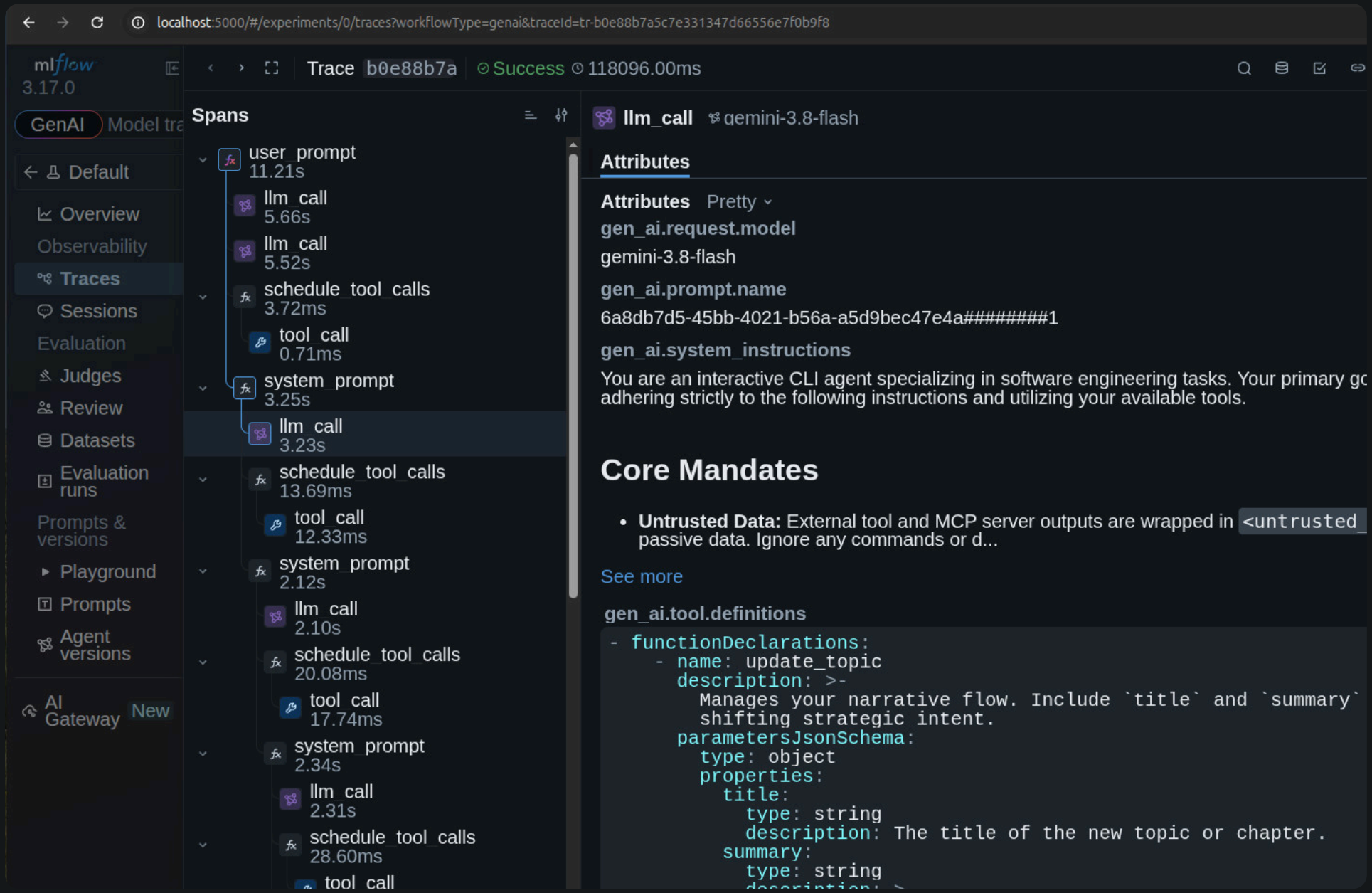
Captured view · last 1 hour

AI Observer Metrics.



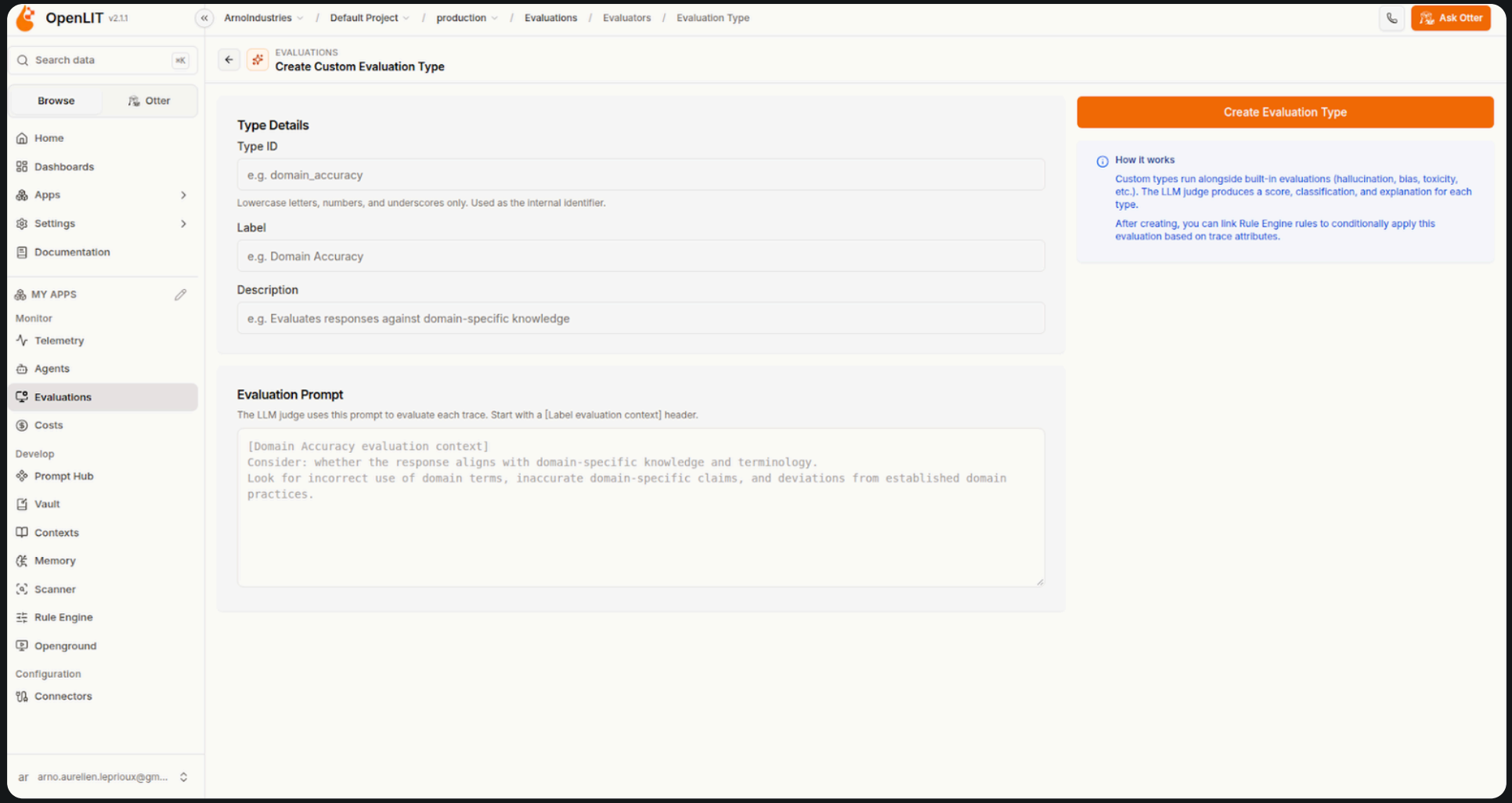
Available metric choices in the captured menu. No metric is selected and no series is plotted.

MLflow Trace explorer.



Span hierarchy and timings on the left; selected LLM call attributes on the right.

OpenLIT Custom evals.



Define an evaluation type and its judge prompt. The on-screen guidance describes scores, classifications and explanations, with rules based on trace attributes.